

Warehouse Module Feature Specification

V1.0 | April 2020 | Derived from Sklat.docx requirements

This document translates the Bulgarian requirements from Sklat.docx into a structured English feature spec, grounded in the actual codebase. Reference screenshots from the original document are embedded inline next to the features they illustrate.

Stack: Next.js 14 App Router | Supabase (direct client) | React Hook Form + Zod | TanStack Query | Tailwind CSS | shadcn/ui | next-intl

Table of Contents

- 1. Warehouse Page & Sidebar Navigation
- 2. Add / Edit Part Modal — Fields
- 3. Cost Price, Sale Price, Margin & Profit
- 4. "Replaced By" Substitute Part Number
- 5. Main List — Column Layout & Row Actions
- 6. Stock Status Indicators (Green / Yellow / Red)
- 7. Inline Quantity Editing
- 8. Offer Integration — Auto-fill from Warehouse
- 9. Offer Integration — Auto-create Part in Warehouse
- 10. Search (Name, Part Number, Substitute Number)
- 11. Search State Persistence
- 12. Export / Import via Excel
- 13. Transfer Offer Data Button
- DB. New Database Table: warehouse_parts
- → Implementation Order
- → Pre-Completion Checklist

1. Warehouse Page & Sidebar Navigation

Field / Setting	Detail
New route	app/[locale]/(admin)/mb-admin/warehouse/page.tsx
Sidebar file	components/admin/AdminSidebar.tsx — add to navItems[] after Earnings
Nav label	admin.sidebar.warehouse → 'Warehouse' / '■■■■■■'
Access	All roles (admin, reception, mechanic) — no adminOnly / superAdminOnly flag
Icon	Box/warehouse SVG — path d="M20 7l-8-4-8 4m16 0l-8 4m8-4v10l-8 4m0-10L4 7m8 4v10M4 7v10l8 4"

2. Add / Edit Part Modal — Fields

The warehouse page has an 'Add Part' button (top right). It opens a modal dialog modelled after AddEditPartModal in components/admin/forms/PartsFieldArray.tsx:124. The same modal is reused for editing an existing part.

Field / Setting	Detail
Part Name	Text — required. DB: warehouse_parts.name
Part Number	Text — required. DB: warehouse_parts.part_number
Manufacturer	Text — default 'MERCEDES' (mirrors PartsFieldArray.tsx:139). DB: manufacturer
Quantity	Integer ≥ 0 . DB: warehouse_parts.quantity
Cost Price	Decimal ≥ 0 (EUR). DB: warehouse_parts.cost_price
Sale Price	Decimal ≥ 0 (EUR). DB: warehouse_parts.sale_price
Replaced By	Text optional. DB: warehouse_parts.replaced_by. See Feature 4.
Margin % (RO)	Auto-computed read-only. Updates live. See Feature 3.
Profit EUR (RO)	Auto-computed read-only. Updates live. See Feature 3.

Tech: Use shadcn Dialog / Input / Label / Button. Validate on confirm with warehousePartSchema (lib/schemas/warehouse.ts). State pattern: local useState per field, same as AddEditPartModal.

Добави част

×

Име на продукт *

Име на продукт

Производител

№ част

MERCEDES

№ част

К-во *

Цена/бр € (с ДДС) *

−

1

+

0

OK

Отказ

Reference screenshot: Add Part modal layout — adapt and extend with Cost Price, Sale Price, Margin % and Profit EUR fields.
Manufacturer defaults to MERCEDES.

3. Cost Price, Sale Price, Margin & Profit

Calculation	Formula	Display example
Margin %	$((\text{sale} - \text{cost}) / \text{cost}) \times 100$	'23.5%' — 1 decimal
Profit EUR	sale - cost	'€4.20' — 2 decimals
cost_price = 0	Margin % → '—' (no division by zero)	Profit still shows raw diff
In modal	Two read-only fields side-by-side	Recalculate on every keystroke
In list	Margin column: '23.5%' line 1	'€4.20' smaller text line 2
DB storage	Only cost_price + sale_price stored	Margin/Profit computed client-side

■ Guard against division by zero: if cost_price is 0 or empty, show '-' for Margin %. Never render Infinity or NaN to the user.

4. "Replaced By" — Substitute Part Number

Field / Setting	Detail
DB column	warehouse_parts.replaced_by TEXT nullable
UI	Optional input in Add/Edit modal, below Part Number field
Search	Searching by this value will find the part (Feature 10)
List display	Not a primary column — visible only inside the edit modal
Example	Part A-001 is replaced by A-002 → replaced_by = 'A-002'

5. Main List — Column Layout & Row Actions

Column order is fixed by the requirement:

Name	Part #	Manufacturer	Qty	Cost	Sale	Margin (% + €)	Status	Date	Actions
Field / Setting		Detail							
Edit action		Pencil icon → opens WarehousePartModal pre-filled with part data							
Qty edit		Click qty number → QuantityPopover (Feature 7)							
Delete action		Trash icon → confirmation dialog → hard delete							
Date column		created_at formatted as DD.MM.YYYY							
Reference file		components/admin/offers/OffersTable.tsx — follow row/grid patterns							
Header		Sticky: sticky top-0 bg-mb-anthracite z-10							

Фильтры

Поиск по №, клиент, ремонт, авто...

Всего статусов

Избранные даты

Избранные даты

№	Оферта №	Клиент	Автомобиль	VIN номер	Рег. номер	Наименование на ремонт	Статус	Сумма	Дата	Действия
1	463217561	Алуря	GLE (V167) GLE 450 4MATIC	4JGF85KX2LA283222	CB488XCE	Смыв полуоси, зазоры амортизаторы	Завершена	4454.00 руб. 8716.27 руб.	03.04.2026	⊕
2	6663499004	Надежда Надеждича	C-Class (W208) C 220 s Wagon	WK206216F0F019182	-	Техническое обслуживание двигателей и задний диф	Черновик	0.00 руб.	03.04.2026	⊕
3	5887258986	Антон Нафедов	A-Class (W177) A250e	WK1K7786L250863	CB7092XP	Техническое обслуживание в DCT 7241	Завершена	672.00 руб. 134.83 руб.	03.04.2026	⊕
4	3743220635	Теодор Геноя	E-Class (W212) E 43 AMG 4MATIC+	WDDZF68BXLA736939	-	Техническое обслуживание 90 Topoil	Идет работа	1900.00 руб. 7112.30 руб.	02.04.2026	⊕
5	2603096646	Теодор Геноя	E-Class (W212) E 53 AMG 4MATIC+	WDDZF68BXLA736939	-	Обслуживание двигателя, кули, свечи и сцеп. тросы	Идет работа	1140.00 руб. 2229.89 руб.	02.04.2026	⊕
6	4502127488	Николай Грозов	GLC (X253) GLC 300 4MATIC	WDD04K8BA1Y00019	1	Обслуживание масла и фильтры двигателя	Завершена	444.00 руб.	03.04.2026	⊕

Reference screenshot: Match this column width, row height, font size, and spacing style exactly. This is the visual target for the warehouse parts list.

6. Stock Status Indicators

Status	Color	Condition	Tailwind classes
In Stock /	Green	qty > 3	bg-green-500/20 text-green-400 border-green-500/30
Limited /	Yellow	qty 1–3	bg-yellow-500/20 text-yellow-400 border-yellow-500/30
Out of Stock /	Red	qty = 0	bg-red-500/20 text-red-400 border-red-500/30

Tech: Create components/admin/warehouse/StockStatusBadge.tsx. Derive status from qty prop. Use Badge from components/ui/badge.tsx. Follow OfferStatusBadge.tsx structure exactly — same glowing badge style, not plain text.

7. Inline Quantity Editing

Clicking the qty number in a row opens the QuantityPopover component (currently at components/admin/forms/PartsFieldArray.tsx:40–122). The requirement explicitly says 'same as in offers'. Extract it to a shared location so warehouse can reuse it.

Field / Setting	Detail
Extract to	components/admin/shared/QuantityPopover.tsx
Trigger	Click qty number in Qty column
Controls	[–] [number input] [+] [✓ confirm]
Save	On confirm: supabase UPDATE warehouse_parts SET quantity=N WHERE id=X
Min	1 via popover (existing logic)
Keyboard	Enter = confirm, Escape = close (already in QuantityPopover)
Toast	useNotification().showSuccess('Quantity updated')

8. Offer Integration — Auto-fill from Warehouse

Field / Setting	Detail
Trigger	Typing in partNumber field inside AddEditPartModal (offer context), ≥2 chars, 300ms debounce
Query	supabase.from('warehouse_parts').select('*').ilike('part_number', '%{term}%').limit(10)
Dropdown	Absolute-positioned list under input: z-50, bg-mb-anthracite, rounded-lg, shadow-xl
On select: name	setDescription(match.name)
On select: price	setUnitPrice(match.sale_price); setPriceInput(String(match.sale_price))
Info badge	Below part# input: 'Stock: {qty} Margin: {pct}%' in matching status color
User can edit	Price remains editable after auto-fill — do not re-override
No match	No auto-fill; normal flow (triggers Feature 9 on save)

■ Stock quantity and margin shown in the quote are **DISPLAY ONLY** — never save them into the quote record. They are hints to help the user decide on pricing.

9. Offer Integration — Auto-create Part in Warehouse

When a part added to an offer has no warehouse match (by part_number), a warehouse entry is silently created with quantity=0, cost=0.

Field / Setting	Detail
Trigger	onConfirm in AddEditPartModal after append/update, when part_number is non-empty
Check	supabase SELECT id FROM warehouse_parts WHERE part_number = exact match
If no match	INSERT: name, part_number, manufacturer='MERCEDES', quantity=0, cost_price=0, sale_price=unitPrice
If exists	Skip — do nothing. Never create duplicates.
Errors	catch silently — never block the offer save
No toast	Silent operation — no user notification

■ Only fires when part_number is non-empty. Empty part_number = no warehouse entry created. ■ Must run after the offer line is saved — never block or delay the offer save flow.

10. Search

Field / Setting	Detail
Fields searched	name, part_number, replaced_by — all three in parallel
Query	.or('name.ilike.%{q}%',part_number.ilike.%{q}%',replaced_by.ilike.%{q}%')
Debounce	300ms
Clear button	X inside input — clears search, resets to full list
Empty state	'No parts found' message with Add Part CTA
Transliteration	Apply getSearchVariants() from hooks/useOffers.ts for Cyrillic/Latin
Pagination	50 per page (larger than offers — parts catalog is bigger)

11. Search State Persistence

Search term persists when navigating away and back, using sessionStorage — same pattern as OfferFilters.tsx (FILTERS_SESSION_KEY).

Field / Setting	Detail
Storage key	WAREHOUSE_FILTERS_KEY = 'mb_warehouse_filters'
On mount	Read sessionStorage; pre-fill search input and re-run query
On change	Write search + page to sessionStorage on every change
Clear on nav	clearWarehouseFilters() called from AdminSidebar Warehouse link onClick
Function export	Export clearWarehouseFilters() from warehouse page; import in AdminSidebar.tsx

■ Test this flow explicitly: search → open part → back → verify search is restored. Do not rely on component state alone — state is lost on navigation.

12. Export / Import via Excel

Export

Field / Setting	Detail
Trigger	'Export' button on warehouse page
Library	xlsx (SheetJS) — npm install xlsx
Format	.xlsx, one sheet named 'Warehouse'

Columns	name, part_number, manufacturer, quantity, cost_price, sale_price, replaced_by, created_at
File name	warehouse_backup_YYYY-MM-DD.xlsx
Client-side	No server route; URL.createObjectURL in browser

Import

Field / Setting	Detail
Trigger	'Import' button → hidden file input, accept='.xlsx,.csv'
Parse	Read first sheet with xlsx; detect columns by header name (case-insensitive)
Upsert logic	supabase.from('warehouse_parts').upsert(rows, {onConflict: 'part_number'})
Required cols	name + part_number (minimum; others default to 0 / 'MERCEDES')
Preview	Show modal with first 10 rows before committing
Validation	Skip rows with empty name or part_number; count skipped
Toast result	'X parts imported, Y rows skipped'
Safety	NEVER deletes existing parts not present in file — upsert only

Tech: Upsert via Supabase .upsert() is sufficient — no dedicated API route needed. Handle malformed files gracefully — show a clear error message, never expose raw exceptions.

13. Transfer Offer Data Button

Field / Setting	Detail
Button location	Top action bar of offer edit page — near Save/PDF buttons in CreateOfferFormV2.tsx
Modal content	Text input: target offer number (10 digits). Typeahead search by offer_number.
Preview	Show: 'Offer #XXXXXXXXXX — [car] — [customer]' before confirm
Transferred	offer_items (parts) + service_actions (labor) — APPENDED to target
NOT transferred	customer_name, phone, car details, notes, discounts, prepayments
Sort order	New items get sort_order = max(existing in target) + 1, 2, 3 ...
Source offer	Unchanged — nothing is removed or modified
Same offer	Validation error: 'Cannot transfer to the same offer'
Toast success	'Parts and labor transferred to offer #XXXXXXXXXX'

Tech: Modify components/admin/forms/CreateOfferFormV2.tsx. On confirm: INSERT copies of current offer_items into target offer_id. INSERT copies of current service_actions into target offer_id.

■ Validate target offer exists before attempting transfer. Show error if not found. ■ A quote must NOT be transferable to itself — validate and block this case explicitly.

DB. New Database Table: warehouse_parts

One new table is required. All existing tables remain unchanged. The table follows the same RLS pattern as supabase/schema.sql.

```
CREATE TABLE warehouse_parts ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), name TEXT NOT NULL, part_number TEXT NOT NULL, manufacturer TEXT NOT NULL DEFAULT 'MERCEDES', quantity INTEGER NOT NULL DEFAULT 0, cost_price NUMERIC(10,2) NOT NULL DEFAULT 0, sale_price NUMERIC(10,2) NOT NULL DEFAULT 0, replaced_by TEXT, created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW() ); CREATE UNIQUE INDEX warehouse_parts_part_number_idx ON warehouse_parts (part_number); CREATE INDEX warehouse_parts_created_at_idx ON warehouse_parts (created_at DESC); ALTER TABLE warehouse_parts ENABLE ROW LEVEL SECURITY; CREATE POLICY "Authenticated users can manage warehouse" ON warehouse_parts FOR ALL TO authenticated USING (true) WITH CHECK (true); CREATE TRIGGER warehouse_parts_updated_at BEFORE UPDATE ON warehouse_parts FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```

Tech: Add types to types/database.ts: WarehousePart, WarehousePartInsert, WarehousePartUpdate.

Implementation Order

#	Task	Files
1	DB migration	supabase/schema.sql + Supabase SQL editor
2	TS types	types/database.ts
3	Zod schema	lib/schemas/warehouse.ts (new)
4	Data hook	hooks/useWarehouseParts.ts (new)
5	Extract QuantityPopover	components/admin/shared/QuantityPopover.tsx (new); update PartsFieldArray.tsx import
6	StockStatusBadge	components/admin/warehouse/StockStatusBadge.tsx (new)
7	WarehousePartModal	components/admin/warehouse/WarehousePartModal.tsx (new)
8	Warehouse page	app/[locale]/(admin)/mb-admin/warehouse/page.tsx (new)
9	Navigation	components/admin/AdminSidebar.tsx messages/en.json messages/bg.json
10	Offer auto-fill	components/admin/forms/PartsFieldArray.tsx (modify AddEditPartModal)
11	Offer auto-create	components/admin/forms/PartsFieldArray.tsx (modify onConfirm)
12	Transfer Data	components/admin/forms/CreateOfferFormV2.tsx + TransferDataModal (new)
13	Export / Import	warehouse/page.tsx + hooks/useWarehouseParts.ts

Pre-Completion Checklist

- 1. warehouse_parts table in Supabase: all columns, unique index on part_number, RLS enabled
- 2. WarehousePart TypeScript interface in types/database.ts
- 3. lib/schemas/warehouse.ts — Zod schema validates all fields
- 4. hooks/useWarehouseParts.ts — list, create, update, delete, upsert, export all implemented
- 5. QuantityPopover extracted to components/admin/shared/QuantityPopover.tsx; import updated in PartsFieldArray.tsx
- 6. StockStatusBadge: qty>3=green, qty 1–3=yellow, qty=0=red
- 7. WarehousePartModal: all 7 input fields + live Margin% + live Profit EUR
- 8. Manufacturer defaults to 'MERCEDES' in Add modal; Edit modal shows stored value
- 9. Margin shows '—' when cost_price = 0; never shows Infinity or NaN
- 10. Warehouse page accessible at /[locale]/mb-admin/warehouse/
- 11. Column order: Name | Part# | Manufacturer | Qty | Cost | Sale | Margin | Status | Date | Actions
- 12. Inline qty edit: click qty number → QuantityPopover → confirm saves to DB → badge updates immediately
- 13. Search across name + part_number + replaced_by with 300ms debounce
- 14. Search state persisted in sessionStorage key 'mb_warehouse_filters'
- 15. clearWarehouseFilters() called from Warehouse sidebar link onClick
- 16. Export: valid .xlsx file, correct columns, named warehouse_backup_YYYY-MM-DD.xlsx
- 17. Import: preview modal → confirm → upsert → toast 'X imported, Y skipped'
- 18. Import never deletes existing parts
- 19. Sidebar 'Warehouse' link visible to all roles; active highlight works
- 20. Translation keys in both messages/en.json and messages/bg.json
- 21. Offer part-number input shows warehouse dropdown (≥2 chars, 300ms debounce)
- 22. Selecting dropdown result fills Name + Sale Price + shows stock/margin badge
- 23. New offer part with non-empty part_number not in warehouse → auto-creates (qty=0, cost=0)
- 24. Auto-create error is silently caught; offer save is never blocked
- 25. 'Transfer Data' button on offer edit page (CreateOfferFormV2.tsx)
- 26. Transfer: appends offer_items + service_actions to target offer
- 27. Transfer: does NOT copy customer/car/notes/discounts/prepayments
- 28. Transfer: source offer unchanged after transfer
- 29. Transfer to same offer: validation error shown, transfer blocked